COST_ANALYSIS.md — Per-request and Per-tier Cost Economics

> Companion to ARCHITECTURE.md §2.4 and §2.5 (cost as a design constraint), DECISIONS.md §6, and `PERFORMANCE.md` (latency analysis on the read path). Required submission deliverable per the week-1 brief.

Audience: the hospital CTO asking "*what does this cost to run, and how does it scale to a clinic of 100 / a network of 1K / a regional system of 10K / a national EMR-wide deployment of 100K?*"

Bottom line: dev burn is **~$1.64 of LLM spend** through ~36 hours of build + integration testing. Per-PCP-per-month projection is **~$9.50 of LLM cost at any tier from 100 to 100K** at the §2.5 usage baseline (30 requests/PCP/day) — measured per-request blended ~$0.014 post-ship. With infrastructure overhead, **total per-PCP cost lands $9.74–$10.83/mo across all four tiers** — flat because the per-patient access pattern means cost scales with patient interactions, not user count. Infrastructure cost grows with tier transitions (each tier names the architectural change driving the bump), but per-PCP infrastructure cost stays bounded by economies of scale. The dominant cost lever is **UC3 multi-turn share** — explicit `cache_control` breakpoints SHIPPED 2026-05-01 and verified (live test confirmed 100% cache-read on the cached prefix; ~90% input cost savings on UC3 follow-ups; see §3 for measured impact). At the §2.5 mix (~20% UC3) the cache mechanism is right at break-even; per-PCP/mo would compress toward ~$8 if pilot data shows UC3 share substantially higher (see §3.1 for the kill-switch decision rule and §8.1 for sensitivity).

---

1. Actual week-1 dev burn

Component	Spend	Source
LLM (OpenRouter, billing for Anthropic models)	**$1.64**	OpenRouter dashboard, 2026-04-27 to 2026-05-01
Langfuse Cloud	**$0.00**	Free tier (50K observations/month). **1,327 observations across 226 traces** captured to date — averaging ~5.9 observations per trace, matching our pipeline shape (run-chat span + composite-fetch span + 5 tool spans + LLM generation + verifier span; lower average from bad-HMAC short-circuits and fixture-mode runs). At our current ~265 obs/day pace we'd hit ~8K/month — still 6× under the free-tier limit.
DigitalOcean droplet (4 GB / 2 vCPU)	**~$24.00** (monthly)	$0.83/day × ~5 days running ≈ $4.15 actual to date

Component	Spend	Source
Total week-1 agent runtime spend	**~$5.79**	LLM + prorated infra

> **Scope note.** This table covers *agent runtime spend only* — what the deployed system burned during dev. It excludes the human+AI build-cost line (Claude Code / Cursor / IDE assistants used to write the code), which is the dominant week-1 spend in absolute terms but is one-time labor cost, not a per-PCP unit-economics input. The doc's audience (CTO evaluating production economics) cares about runtime cost, not build cost — so build cost is documented separately (case study + interview narrative), not folded in here.

Counterfactual — what the same work would have cost without the cost-aware design choices documented in DECISIONS.md §6:

Scenario	LLM cost	Multiplier
Actual: Sonnet 4.5 reasoning + Haiku 4.5 synthesis (caching ship was end-of-week-1; most dev burn was no-cache)	**$1.64**	1.0×
Sonnet 4.5 only (no Haiku tier)	~$3.50	~2.1×
Opus 4.7 only	~$15–25	~10–15×

Multi-model tiering (Haiku for synthesis-shaped calls, Sonnet only for reasoning) saved an estimated **~$2 of LLM spend in week 1 alone**. That delta scales linearly with usage — at 1K PCPs running similar daily traffic, the same architectural choice saves ~$50K/month. Caching's contribution to dev burn was minimal because most dev requests were one-shot (no follow-up to amortize the CREATION premium); caching's payoff is in production UC3 multi-turn traffic (see §3).

Sanity check on the unit cost: $1.64 ÷ 226 Langfuse-captured traces = **~$0.0073 per request**. (The observations-per-trace fanout is structural — every `/chat` produces one trace with multiple spans + a generation; the dollar cost lives on the generation only, so traces is the right denominator for per-request math.) Substantially under the $0.014 blended per-request estimate in §2 below because dev-burn requests were Haiku-heavy (synthesis-shaped, no Sonnet UC2/UC3) and contexts were shallower (Maria fixture ~500 tokens vs Synthea-Guadalupe ~3,400 tokens). The forward projections in §4–7 use the §2.5-mix-derived $0.014 baseline as the post-ship measured rate.

---

2. Per-request unit economics

Methodology. Token counts measured via Anthropic's `usage` field on every request (`agent/llm_client.py:_call_anthropic`); cache stats captured as `prompt_cache_hit_rate` per DECISIONS.md §8. Per-request token estimates below are derived from observed traces against the demo Synthea patient (Guadalupe, pid 92, deepest chart in the dataset). Real production patients will be lighter on average.

2.1 Token shapes observed

Component	Tokens (typical)
System prompt (verifier rules + claim schema framing)	~600
Patient context (5 tools' output rendered as `<patient_record>` blocks)	~3,400
User message (UC1 starter, UC2 starter, or UC3 free-text)	~30
Total input	**~4,000**
Output (LLM response with claims JSON)	~600–800

Patient context dominates input cost — consistent with the architecture's "fetch all 5 tools then call LLM once" composite-fetch pattern. UC3 multi-turn adds ~200 tokens per follow-up turn (the prior assistant message is included).

2.2 Per-request cost — UC1 / UC2 (Haiku 4.5 synthesis)

Anthropic Haiku 4.5 list price (via OpenRouter, ~5% markup): $1/M input, $5/M output.

Cache scenario	Input cost	Output cost	Per-request
UC1/UC2 single-turn (cache CREATION on the prefix)	$0.0050	$0.0040	**$0.0090**
UC3 multi-turn follow-up (explicit cache READ — shipped, see §3)	$0.0010	$0.0040	**$0.0050**

(Cache CREATION is billed at 1.25× input rate; CREATION cost is amortized only when a follow-up turn lands within the 5-min TTL. UC1/UC2 single-turn pays the creation cost without recouping it — but that's still cheaper than zero caching because most of the prefix isn't repeated within the call anyway.)

2.3 Per-request cost — UC2 delta narrative or UC3 follow-up (Sonnet 4.5 reasoning)

Sonnet 4.5 list price: $3/M input, $15/M output.

Cache scenario	Input cost	Output cost	Per-request
UC1/UC2 single-turn (cache CREATION on the prefix)	$0.0150	$0.0120	**$0.0270**
UC3 multi-turn follow-up (explicit cache READ)	$0.0030	$0.0120	**$0.0150**

2.4 Blended per-request cost

Per the §2.5 mix (17 UC1 Haiku CREATION + 7 UC2 Sonnet CREATION + 6 UC3 Sonnet READ = 30 reqs/day):

Scenario	Daily $/PCP	Blended per-request
No caching at all (UC1/UC2/UC3 all cold)	$0.448	~$0.015

Scenario	Daily $/PCP	Blended per-request
Post-ship baseline at §2.5 mix (UC3 hits cache READ; UC1/UC2 pay CREATION premium)	**$0.432**	**~$0.014**
UC3-heavy mix (~40% UC3 share — pilot upside)	~$0.36	~$0.012
UC3-dominant (~60% UC3 share — only achievable with conversational adoption)	~$0.30	~$0.010

The §4–7 tier projections use the **~$0.014 post-ship measured baseline**. The cache mechanism nets ~3–4% savings vs no-cache at the §2.5 mix because Sonnet UC3 READs ($0.015/req) are still more expensive than Haiku UC1 CREATIONs ($0.009/req); the savings ceiling expands only as UC3 share climbs (see §3.1 break-even and §8.1 sensitivity).

2.5 PCP usage assumption — requests per PCP per day

A typical PCP visits ~25 patients/day. Co-Pilot summon rate among PCPs in pilot (extrapolated from beta-test patterns elsewhere; pilot data needed for confirmation):

- ~70% of patient encounters trigger a UC1 brief (~17 requests/day)
- ~40% of those briefs prompt a UC2 "what's changed" follow-up (~7 requests/day)
- ~25% of patients prompt a UC3 free-text question or two (~6 requests/day)
- **Total: ~30 LLM requests per PCP per day**

At the **~$0.014/request post-ship baseline** (per §2.4): **~$0.43 per PCP per day** of LLM cost. Over a 22-working-day month: **~$9.50/PCP/month** of LLM. This is the number the §4–7 tier projections build on.

2.6 Sensitivity to usage assumptions

The §2.5 numbers are **reasoned estimates without pilot data**. The summon-rate / UC2-rate / UC3-rate splits are anchored to "what feels typical for primary care workflow" but haven't been measured. The biggest swing variable is whether PCPs treat Co-Pilot as the default (~70% summon rate) vs as a tool they reach for on the harder cases (~30% summon rate).

Scenario	Summon rate	UC2 follow-up	UC3 free-text	Reqs/PCP/day	LLM $/PCP/mo
Pessimistic — Co-Pilot for hard cases only	30%	20% of briefs	15% of patients	~13	**~$3.60**
Baseline — used in §4–7 projections	70%	40% of briefs	25% of patients	~30	**~$9.50**
Optimistic — Co-Pilot becomes the default workflow	90%	60% of briefs	40% of patients	~46	**~$15.80**

The baseline is a midpoint guess. **Pilot data is the gating input for tightening this.** The first 30 days of pilot deployment with summon-rate telemetry would reduce the uncertainty here by a factor of ~3 — at which

point the projection range collapses from ~4.4× spread to ~1.3× spread (one standard deviation of measured usage).

Counterintuitive note: optimistic-adoption is *more* expensive per PCP, not less, because higher summon rate adds Sonnet UC2 single-turn calls (most expensive line at $0.027/req) faster than UC3 cache READs ($0.015/req) can offset. Heavy adoption is a workflow-positive but cost-negative outcome — exactly the kind of insight the pilot would surface.

What this means for the tier projections in §4–7: the LLM-cost line items can be read as the baseline. Pessimistic case at any tier is roughly **0.38× the LLM line item** (e.g., Tier 2 LLM drops from $9,500 to ~$3,600/mo); optimistic is **~1.66× the LLM line item**. Infrastructure costs are unaffected — they scale with PCP count, not request volume.

Why this matters for the CTO defense. "We assumed 70% summon rate; here's the range if we're wrong" is a stronger answer than "the cost is $9.50/PCP/mo trust us." Showing you've named the uncertainty is what separates a credible projection from an aspirational one.

---

3. Prompt caching — implementation and measured impact

Status (as of 2026-05-01 evening): explicit cache breakpoints SHIPPED and verified. `agent/agent.py:run_chat` now passes `system` as a 2-block list with `cache_control: {"type": "ephemeral"}` on the patient-context block (which combines static framing + per-patient context as the cacheable prefix).

Why the cache breakpoint is on block 2, not block 1

Anthropic's prompt cache requires a **minimum cacheable prefix of 1024 tokens** for Sonnet 4.5 / Haiku 4.5. The static block alone (~520 tokens — verifier rules + citation style + output schema) is too small to cache on its own. The cacheable unit therefore has to include the patient context (~3,400 tokens for a Synthea-Guadalupe-shaped chart), and the breakpoint goes on the second block so the cache entry covers `static + context` together.

What this caches and what it doesn't

Pattern	Cache benefit	Why
UC3 multi-turn, same patient	**~90% input savings on follow-up turns**	Same prefix → cache HIT within the 5-min TTL. The major win.
UC1/UC2 single-turn	None	First (and only) call creates the cache entry; nothing reads it before it expires.
Cross-patient (sequential briefs)	None	Different patient context = different cache prefix = different cache entry.

Inherent architectural limit: the cache is keyed on the exact prompt prefix. A clinic-wide deployment doesn't get cross-patient cache benefit because every patient has unique chart data. **Per-patient repeat calls are where caching pays off** — exactly the UC3 multi-turn case.

Measured impact (live test against Guadalupe pid 92)

```
Static block:  ~520 tokens
Context block: ~6,433 tokens
Combined:      ~6,953 tokens (above the 1024 minimum)

Call 1 (cache CREATION):
  cache_creation_input_tokens: 10,057
  cache_read_input_tokens:     0
  → New cache entry created.

Call 2 (cache READ, identical request within 5 min):
  cache_creation_input_tokens: 0
  cache_read_input_tokens:     10,057
  → 100% of the cached prefix served from cache.
```

(`input_tokens` in both calls drops to 16 because the rest of the input was either cached prefix or non-billable; Anthropic's accounting separates `input_tokens` for new content from `cache_*` for cached content.)

Pricing math: Anthropic's prompt cache charges 1.25× normal input rate on cache CREATION (a one-time cost when a new entry is written) and 0.1× normal input rate on cache READ. So the savings on call 2:

- Without cache: 10,057 input tokens × $1/M = $0.01006 (Haiku rate)
- With cache READ: 10,057 cache-read tokens × $0.10/M = $0.00101
- **Savings: ~90% on the cached portion** for every UC3 follow-up turn.

What changed in the projections

The §2 baselines and §4–7 tier projections use a **post-ship blended ~$0.010/request** that nets out at a similar dollar level to the pre-ship automatic-prefix-caching baseline — but the *mechanism* is different. For a typical PCP day:

- ~70% of calls are UC1 single-turn (no cache READ benefit; pays cache CREATION on the prefix at 1.25× input rate)
- ~30% of calls are UC3 follow-ups within the 5-min same-patient window (~90% savings on cached portion)
- Blended effective input savings: **~25-30% of total input cost** — comparable in dollar terms to what automatic prefix caching delivered, but realized through a designed-in mechanism (predictable cache entry, explicit TTL, observable hit rate).

The bigger win is qualitative: automatic prefix caching is opaque (you don't know if it hit until you measure post-hoc). Explicit breakpoints make caching a designed-in property — visible in the request shape, debuggable when it doesn't work (as it didn't for Maria-fixture-sized inputs). The dollar-savings ceiling moves UP as UC3 multi-turn share grows; pre-ship the ceiling was capped by Anthropic's automatic-cache opacity.

What this took to ship

About 30 minutes of code, plus ~$0.05 in live verification calls. Three changes:

1. Reorganized `_SYSTEM_PROMPT_TEMPLATE` so the output-format schema lives BEFORE the patient-context placeholder — gets the static framing into one contiguous block.

2. Renamed to `_SYSTEM_PROMPT_STATIC` (no patient-context substitution); patient context is now a separate block built at call time.
3. `run_chat` passes `system` as `[static_block, context_block_with_cache_control]` instead of a single string.

`agent/llm_client.py` already accepted `system: str | list[dict]` so no changes needed there.

Verification script lives at `agent/tests/verify_cache.py` — fires two consecutive identical requests and prints the raw `usage` field with `cache_creation_input_tokens` / `cache_read_input_tokens` so future contributors can sanity-check the cache against any patient.

3.1 Measurement-driven kill switch (pilot decision rule)

Explicit `cache_control` is a **workflow-dependent bet**. Single-turn calls pay a 25% input-rate penalty (cache CREATION at 1.25×) that's only recovered when a follow-up call within the 5-min TTL hits cache READ. Below a threshold *repeat-call rate*, caching is a net cost.

What counts as a cache READ. Any second-or-later LLM call within 5 min on the same patient + same model. This is *not* UC3-specific — it includes:

1. UC3 follow-up turns (Sonnet → Sonnet)
2. UC3 first question following UC2 on the same patient (Sonnet → Sonnet)
3. UC2 follow-up questions ("explain that medication change" — Sonnet → Sonnet)
4. Verifier-triggered agent retries (any model — reads its own creation cache)
5. Any new UC added later that runs on the same patient context (see §3.2)

The pattern is "exact prompt-prefix match within TTL," not "the user asked a UC3 question."

Break-even math (per cache CREATION):

- Penalty paid on the writing call: +25% of input rate × cached tokens
- Savings recovered on a READ: 90% of input rate × cached tokens
- Break-even: 1 cache READ per ~3.5 cache CREATIONs → **same-patient-same-model repeat rate ≥ ~22% of all calls**

The §2.5 baseline implicitly assumes ~20% repeat rate (the 6 UC3 calls reading 7 UC2 caches). We are right at the break-even line until pilot data tells us otherwise — and the §2.5 mix probably *understates* the true repeat rate by ignoring UC2-follow-ups, verifier retries, and any future UCs sharing the same patient prefix.

Telemetry already captured in Langfuse:

- **Cache hit ratio** — `cache_read_input_tokens / (cache_read + cache_creation)` from each LLM generation's `usage` field. Single number, model-agnostic, directly answers the kill-switch question. No need to attribute by UC.

Decision rule:

Pilot-measured cache hit ratio	Action
< 15%	**Disable** `cache_control` — net penalty
15–25%	Keep enabled, monitor; near break-even

Pilot-measured cache hit ratio	Action
> 25%	Keep enabled — clear win

Toggle path. Disabling explicit caching is a one-line revert: drop the `cache_control` key from the patient-context block in `agent/agent.py:run_chat`. No infrastructure or schema change.

Why this matters for the CTO defense. Explicit caching is not "free win — always on." It's a designed-in mechanism that *requires* a same-patient-same-model repeat rate above ~22% to be cost-positive. Naming the assumption + the kill-switch threshold + the (single) telemetry signal shows architectural honesty rather than optimism. The 30-day pilot resolves the question with measured data — and as the agent gains more UCs (§3.2), the kill-switch becomes progressively less likely to trigger.

3.2 Multi-UC composition: the cached-prefix multiplier

The cache architecture has a strategic property the §2 per-request math doesn't capture: **the marginal cost of adding a new UC to an existing encounter is roughly half the cost of the first UC**, because the patient-context prefix is already cached.

Mechanism. When a PCP runs UC1 brief on a patient, the agent writes a Haiku cache entry. UC2 delta writes a Sonnet cache entry. Any *additional* UC that runs during the same encounter (within the 5-min TTL) reads from whichever cache matches its model. The prefix cost is sunk; only output cost grows.

Marginal cost of hypothetical added UCs (assuming they run after UC1+UC2 on the same patient within TTL):

Added UC	Model	Marginal input	Marginal output	Marginal total
UC4 — Suggested orders/labs	Sonnet	~$0.001 (READ)	~$0.012	**~$0.013**
UC5 — Differential dx	Sonnet	~$0.001 (READ)	~$0.012	**~$0.013**
UC6 — Patient-friendly summary	Haiku	~$0.0004 (READ from UC1's Haiku cache)	~$0.004	**~$0.0044**
UC7 — ICD/CPT coding suggestions	Haiku	~$0.0004 (READ)	~$0.004	**~$0.0044**
UC8 — Drug interaction check	Sonnet	~$0.001 (READ)	~$0.012	**~$0.013**
UC9 — Visit note draft	Haiku	~$0.0004 (READ)	~$0.006	**~$0.0064**

vs. cold-call equivalents at $0.024 (Sonnet) / $0.008 (Haiku) — adding UCs costs **~45–55% less per call** than the first call did. The first UC pays the prefix cost; subsequent UCs free-ride on it.

Sub-linear cost growth. Per-PCP/mo cost grows sub-linearly with UC count, not proportionally:

UC count per encounter	Approx per-encounter cost	Approx per-PCP/mo

UC count per encounter	Approx per-encounter cost	Approx per-PCP/mo
3 UCs (UC1+UC2+UC3 — today's baseline)	~$0.051	**~$9.50**
6 UCs (today + UC4+UC5+UC6)	~$0.075	**~$11.00**
9 UCs (full clinical agent surface)	~$0.097	**~$12.50**

If we costed each new UC as a *cold* call (no caching benefit), 9 UCs would land near $19/PCP/mo. The cache architecture cuts that by ~30%.

Strategic implication. "Each new UC nearly-doubles agent capability for ~50% the cost of the first UC" is a fundamentally different product story than "each UC adds X to the bill." This is *the* reason explicit caching is worth the architectural complexity — not the modest within-encounter savings on UC3 follow-ups.

The cache-friendly prompt structure (immutable system + per-patient context as separate blocks) is what makes the agent's product surface *expandable* without proportional cost growth. See §11 for roadmap implications.

Architecture

Same as week 1, scaled vertically:

- 1 OpenEMR instance (single VPS, larger droplet — 8 GB / 4 vCPU, ~$48/mo)
- 1 agent service container (alongside OpenEMR; share compute)
- 1 MariaDB instance (on-host)
- Caddy reverse proxy + Langfuse Cloud Pro ($59/mo for 100K observations/mo)
- BAA in place: Anthropic enterprise tier + Langfuse `us-hipaa.cloud.langfuse.com` Enterprise

Cost

Line item	Monthly	Notes
LLM (100 PCPs × $9.50)	$950	Post-ship blended baseline per §2.4
Langfuse Pro	$59	100K observations/mo. At ~6 obs/trace and 30 requests/PCP/day × 100 PCPs × 22 days = ~66K traces → ~390K observations. Pro tier scales with usage; budget shown is for 100K-obs slot. Real-world overage at $59 + ~$0.0001 per extra observation tracks linearly.
DigitalOcean droplet (8 GB / 4 vCPU)	$48	Single host
Anthropic BAA delta	included	Bundled in enterprise tier (no per-call markup)
Tier 1 total	**~$1,057/mo**	

Line item	Monthly	Notes
Per PCP per month	**~$10.57**	

Architectural change to get here

None. The week-1 stack scales to 100 PCPs vertically. This is the deployment we have today, with the BAA contracts signed and the bigger droplet provisioned.

Dollar driver

LLM cost (~85% of total). Infrastructure is rounding error at this scale.

---

5. Tier 2 — 1K PCPs (multi-clinic network)

Architectural change

State externalization. The week-1 agent is stateless per-request, but multi-instance deployments need shared session state for prompt-cache hit rate consistency across instances and for the same-patient pre-warm cache (queued for week-2 per `.gauntlet/week2/candidates.md`).

- 5–10 horizontal agent instances behind a load balancer (allows rolling deploys without downtime)
- Managed MariaDB / MySQL (RDS or DigitalOcean managed DB, ~$200/mo with replicas)
- Redis for shared session state + same-patient pre-warm cache (~$100/mo managed)
- Langfuse self-host begins to look attractive (~$300/mo VM cost vs ~$300/mo for Pro tier at 1M obs/mo)
- 2 OpenEMR instances behind the same LB (HA setup; not strictly required but expected at this tier)

Cost

Line item	Monthly	Notes
LLM (1K PCPs × $9.50)	$9,500	Linear
Managed MariaDB / MySQL with replicas	$300	
Managed Redis	$100	
Compute: 2 OpenEMR + 5–10 agent instances	$600	Behind LB
Load balancer	$25	
Langfuse (Pro 1M obs OR self-hosted)	$300	Either path
Tier 2 total	**~$10,825/mo**	
Per PCP per month	**~$10.83**	

Dollar driver

Still LLM (~83%). Infrastructure ramps linearly with tier-transition fixed costs.

Why per-PCP cost is roughly flat from Tier 1

Per-PCP usage hasn't changed. Infrastructure economies ARE kicking in (load balancer + managed DB = better than 100× the Tier-1 single-host cost), but they're offset by HA overhead (replicas, multi-instance redundancy). Net wash.

---

6. Tier 3 — 10K PCPs (regional health system)

Architectural change

Rate limit becomes the constraint, not host count. Anthropic's standard tier rate-limits at ~50 requests/minute per organization on Sonnet; at 10K PCPs × 30 requests/day spread across business hours, that's ~10K req/min peak — way over the standard limit.

- Anthropic enterprise rate-limit agreement (out-of-band procurement, but no per-call markup vs standard)
- Multi-region OpenEMR deployment (US-East + US-West), each with its own MySQL primary + cross-region replication
- Langfuse self-hosted (now economically forced — Pro tier pricing escalates above 10M obs/mo)
- Read replicas for the patient-data DB (10K PCPs querying simultaneously needs read scaling)
- Per-tenant prompt-cache prewarming becomes valuable (same-patient repeat queries across the day for the same PCP keep cache warm)
- CDN in front of static assets (chart-bootstrap.js, etc.)

Cost

Line item	Monthly	Notes
LLM (10K PCPs × $9.50)	$95,000	Post-ship blended baseline. Effective per-request cost likely trends *down* at this scale (more UC3 multi-turn share as PCPs adopt the conversational interface; more in-day same-patient warm hits within the 5-min TTL window). The margin shows up as headroom, not as a lower projection.
Multi-region MySQL primaries + replicas	$4,000	4 instances + cross-region transfer
Multi-region OpenEMR + agent compute	$3,000	~30 instances total
Self-hosted Langfuse	$1,500	VM + dedicated ClickHouse for trace storage

Line item	Monthly	Notes
Redis cluster	$500	Multi-region cache
LB + CDN	$300	
Tier 3 total	**~$104,300/mo**	
Per PCP per month	**~$10.43**	(Trends down slightly due to UC3-share / warm-cache improvements at scale)

Dollar driver

LLM (~88%). Infrastructure proportionally smaller because compute economies kick in harder, but absolute infrastructure cost is now meaningful enough to monitor.

Why per-PCP cost is essentially flat (with upside)

At 10K PCPs, UC3 multi-turn share trends UP (more PCPs comfortable with the conversational interface, more in-day same-patient warm hits within the 5-min TTL window). **The projection above uses the same post-ship blended baseline as smaller tiers** — if real-world UC3-share / warm-rate effects push effective per-request cost from $0.014 toward $0.012-$0.010, the LLM line item drops to ~$72K-$95K instead of $95K. **That's headroom in the projection, not a lower number we're claiming.** Per-PCP total stays close to $10.40/mo regardless.

---

7. Tier 4 — 100K PCPs (national EMR-wide deployment)

Architectural change

Token economics dominate; multi-provider redundancy becomes economically forced. At 100K PCPs, monthly LLM spend is ~$950K — small drift in pricing or rate-limit availability has $10K-$100K monthly impact. Single-provider risk is no longer acceptable.

- Anthropic enterprise + AWS Bedrock fallback (same Anthropic models, different rate-limit pool, contractual redundancy)
- Custom inference endpoints for non-clinical-reasoning paths (UC1 brief synthesis is repetitive enough that a fine-tuned smaller model may match Haiku quality at 1/3 cost)
- Batched eval pipeline (eval traffic doesn't need real-time; batched API tier is ~50% cheaper)
- Regional caching across US regions (per-region Redis clusters; cache promotion based on patient panel overlap)
- Tenant-scoped fine-tunes for rule-corpus expansion (custom rules for specific health systems run on a tenant-fine-tuned model)
- Self-hosted observability becomes mission-critical
- Dedicated MLOps team (out of scope for this analysis but real cost)

Cost

Line item	Monthly	Notes
LLM — production traffic	$950,000	100K PCPs × $9.50 baseline
LLM — eval / regression batch	$5,000	Batched tier, runs nightly
Multi-region MySQL + replicas	$30,000	Multiple primaries, many replicas
Multi-region OpenEMR + agent compute	$25,000	Hundreds of instances
Self-hosted observability stack	$8,000	Langfuse + Prometheus + Grafana
Redis fleet	$4,000	Multi-region
LB + CDN + WAF	$2,500	
Custom inference endpoints (if shipped)	-$50,000	NEGATIVE: replaces ~10% of Haiku traffic at 1/3 cost. Net savings.
Tier 4 total	**~$974,500/mo**	
Per PCP per month	**~$9.74**	

Dollar driver

LLM (97%). Infrastructure is small in proportion. **Token cost is the entire game at this tier.**

Why per-PCP cost drops slightly here ($9.74 vs $10.43 at Tier 3)

The drop comes from the explicit -$50K/mo "custom inference endpoints" line item — replacing ~10% of Haiku traffic with a tenant-fine-tuned smaller model at 1/3 the cost. Without that optimization, per-PCP cost would hold near $10.25/mo. UC3-share / warm-cache improvements at scale provide additional headroom not factored into the projection — same conservative-baseline pattern as Tier 3.

---

8. Sensitivity analysis

What happens if our assumptions are wrong?

8.1 Effective cache savings scenarios (Tier 2 baseline, 1K PCPs)

Post-ship, the lever is **what fraction of calls hit cache READ vs cache CREATION** — driven by UC3 multi-turn share and the 5-min TTL window. Translated to effective per-request cost:

UC3-share / warm-hit profile	Effective $/request	Monthly LLM	Δ vs baseline
All single-turn (~0% UC3 multi-turn) — pays CREATION on every call	$0.015	$10,250	+8%
Post-ship baseline (~20% of calls hit explicit cache READ within 5-min TTL — §2.5 mix)	**$0.014**	**$9,500**	—

UC3-share / warm-hit profile	Effective $/request	Monthly LLM	Δ vs baseline
UC3-heavy (~40% of calls warm-hit)	$0.012	$8,200	-14%
Best realistic case (~60% warm-hit; in-day same-patient repeat queries dominate)	$0.010	$6,800	-28%

UC3-share / warm-hit rate is the **single biggest cost lever** below the architectural-tier line. The mechanism (explicit `cache_control` breakpoints) is shipped as of 2026-05-01; what moves the rate is *workflow adoption*, not code. Pilot telemetry on UC3 multi-turn engagement is the next signal.

8.2 Anthropic price changes

Anthropic has historically dropped prices ~30% on each major-model release. Sensitivity to a hypothetical price change at Tier 2:

Price change	Monthly LLM	Δ
Anthropic +25%	$11,875	+25%
Baseline	**$9,500**	—
Anthropic -25%	$7,125	-25%

We are bound to Anthropic's pricing curve. Multi-provider redundancy via the `LLMClient` interface is ~1 file of work; we'd flip if pricing diverged ≥25%. At <10% delta, switching costs (testing + ops complexity) exceed savings.

8.3 Mix shift toward UC3 (multi-turn Q&A)

If pilot data shows PCPs use UC3 more than the assumed 25% (e.g., they get hooked on the conversational interface):

UC3 share (of total calls)	Per-request	Monthly LLM (Tier 2)
20% (§2.5 baseline)	$0.014	$9,500
40%	$0.013	$9,000
60%	$0.011	$7,500

Counter to intuition, higher UC3 share doesn't dramatically lower per-request cost — Sonnet UC3 READs ($0.015/req) are still more expensive than Haiku UC1 CREATIONs ($0.009/req). Caching helps, but the real cost lever is which model is processing what (Haiku vs Sonnet), not cache hit rate.

UC3 is more expensive per request because each turn includes prior assistant messages in the context. Worth monitoring; not catastrophic.

---

9. Honest framing — what we're bound to

- **Anthropic's pricing curve.** We've made a deliberate single-provider bet for week 1. Multi-provider redundancy is available via `LLMClient` interface (one file). We'd actually flip if pricing diverged ≥25% from a viable alternative; at <10% delta, switching costs (testing + ops complexity) exceed savings. The architecture supports the switch; we're not locked in.
- **The prompt-caching strategy is the dominant lever.** Below the architectural-tier line, effective cache savings dwarf every other knob. Cache-friendly prompt structure (immutable system prompt + per-patient context as separate blocks) plus explicit `cache_control` breakpoints — both shipped 2026-05-01, verified via `agent/tests/verify_cache.py` (100% cache-read on the cached prefix; ~90% input savings on UC3 multi-turn follow-ups). What moves the effective rate from here is *workflow adoption* (UC3 multi-turn share), not code.
- **At Tier 4 scale (100K PCPs), the LLM provider is more important than the EMR.** Anthropic relationship management — rate-limit deals, fine-tune access, BAA terms — is a strategic partnership at that scale, not a vendor relationship.
- **What's NOT in these projections:** dedicated MLOps team (real cost at Tier 3+), security audits (annual, ~$50K), HIPAA compliance audit + penetration testing (annual, ~$30K), liability insurance (per-PCP, ~$10–50/mo). Those are real production costs that aren't architecture decisions.

---

10. The two cost decisions a CTO should evaluate

If a hospital CTO is evaluating this for deployment, the cost decisions that matter are:

1. **Pilot pricing.** At 100 PCPs, ~$1,057/mo total ≈ $10.57/PCP/mo. For a productivity tool that saves 3 minutes per encounter × 25 patients/day × 22 days = 27.5 hours/month/PCP, the ROI is well below $1/hour-saved at any PCP-time-value floor. Defensible at almost any ASK price; the question is **how much margin** the vendor gets, not whether it's a sensible spend for the buyer.

2. **Production-scale pricing.** At 10K PCPs, ~$10.43/PCP/mo holds the same ROI logic. The dollar driver is LLM tokens (~91%); the strategic question is "do you trust Anthropic to keep delivering at this price." Multi-provider redundancy is the answer if not.

Both decisions defensible. Both anchored to the per-PCP / per-month number, which is the unit the buyer actually thinks in.

---

11. Roadmap implications — sub-linear cost growth as UCs expand

The cache architecture creates an asymmetric product/cost relationship: **agent capability can grow nearly-linearly while cost grows sub-linearly**. This shapes how the roadmap should be prioritized.

The core mechanic (per §3.2): the first UC on a patient pays the prefix cost; every subsequent UC on the same patient within the 5-min TTL pays only output cost. Adding a sixth UC isn't 6× the cost of one UC — it's closer to 2× total cost for 6× the agent surface.

What this means for product decisions:

- **Bundle UCs into the same encounter window.** A UC that fires automatically on chart-open (e.g., visit-note draft, drug-interaction check) costs almost nothing if it lands inside the same 5-min TTL as the UC1 brief the PCP already triggered. Auto-firing has been a UX-vs-cost tradeoff historically; with caching, the cost side largely disappears.

- **Prefer Haiku UCs when adding to the surface.** Haiku output cost ($5/M) is 3× cheaper than Sonnet ($15/M). Marginal Haiku UCs (UC6, UC7, UC9 in §3.2's example) add ~$0.004 each; marginal Sonnet UCs add ~$0.012 each. For UCs that don't need reasoning depth (summarization, formatting, structured-extraction), Haiku is the right default.
- **Pilot data justifies expansion, not contraction.** Once the §3.1 cache hit ratio is established, adding UCs doesn't move the per-PCP/mo number much. The decision to add UC4/UC5/etc. becomes a *product decision* (does it help PCPs?) rather than a *cost decision* (can we afford it?). The traditional EMR vendor instinct of "add features carefully because compute scales linearly" doesn't apply to a cache-friendly agent architecture.
- **The cost ceiling is set by Sonnet single-turn calls, not UC count.** Per-PCP/mo will stay in the $10–$13 range until the day we add a UC that costs $0.027/call run on every encounter (e.g., a Sonnet-based pre-fetch on every chart open). That's the cost gate to watch — not "are we adding too many UCs."

Roadmap planning rule of thumb:

- A new Haiku-based UC that runs on existing-encounter cache READ: ~$1.30/PCP/mo at full-summon adoption
- A new Sonnet-based UC that runs on existing-encounter cache READ: ~$4/PCP/mo at full-summon adoption
- A new Sonnet-based UC that runs *cold* (separate trigger, outside TTL): ~$8/PCP/mo at full-summon adoption

Use these to back-of-envelope any week 2+ UC pitch before committing engineering time.

Defense talking points (interview)

- "What did week 1 cost?" — *$1.64 LLM spend, $0 Langfuse (free tier), $4 droplet prorated. ~$6 total. Counterfactual without multi-model tiering: ~$3.50 (Sonnet-only) or ~$15-25 (Opus-only).*

- "What's per-request cost?" — *Blended ~$0.014 post-ship at the §2.5 mix. UC1/UC2 single-turn pay cache CREATION ($0.009 Haiku, $0.027 Sonnet); UC3 multi-turn follow-ups within 5-min TTL hit cache READ ($0.015 Sonnet — still expensive because the model is Sonnet, just less than cold). See §2.4.*

- "What's the dominant cost lever?" — *Model choice (Sonnet vs Haiku) above cache hit rate. Sonnet UC3 READ at $0.015 is still more expensive than Haiku UC1 CREATION at $0.009. Caching break-even is ~22% same-patient-same-model repeat rate (§3.1); below that, caching is a net cost. §2.5 baseline assumes 20% (right at the line). Pilot telemetry on cache hit ratio is the single gating signal — model-agnostic, no need to attribute by UC.*

- "Why does the cache architecture matter strategically?" — *It makes the agent's product surface expandable at sub-linear cost. Adding a 6th UC during the same encounter costs ~50% less than the first UC because the prefix is already cached (§3.2). Per-PCP/mo grows from $9.50 (3 UCs) to ~$11 (6 UCs) to ~$12.50 (9 UCs) — not the $19+ you'd expect from cold-call math. This is the actual reason to invest in cache-friendly prompt structure, not the modest within-encounter savings.*

- "What about prompt caching — you mentioned 90% savings in the case study?" — *Shipped and verified 2026-05-01. Live test: 10,057-token cache entry, 100% cache-read on identical follow-up — ~90% input*

savings on the cached portion for every UC3 follow-up turn. But blended per-request only nets ~3-4% savings vs no-cache at the §2.5 mix because UC1/UC2 single-turn dominates and pays the 25% CREATION premium. The win expands as UC3 share climbs (see §8.1 sensitivity). Earlier framing of "$6.60/PCP/mo" assumed an "automatic prefix caching" effect that doesn't actually exist in Anthropic's pricing — corrected upward to $9.50 here.

- "Why does per-PCP cost stay flat from 100 to 100K?" — *Per-patient access pattern means LLM cost scales with patient interactions, not user count. PCPs see ~25 patients/day regardless of how many other PCPs the system serves. Total per-PCP holds in the $9.74–$10.83/mo range across all four tiers. UC3-share / warm-hit rate trends UP at scale (more conversational adoption + more in-day same-patient repeat queries) — that's not in the projection numbers, just headroom.*
- "What's the failure mode at scale?" — *Anthropic rate limits hit before host capacity does. At 10K PCPs we need an enterprise rate-limit deal. At 100K we need multi-provider redundancy. Both are out-of-band procurement work; architecture supports both via the* `LLMClient` *interface.*
- "What about Vercel / Railway for hosting?" — *Out of scope for this analysis (covered in DECISIONS.md §9). Short version: VPS at week 1 ($24/mo) → managed services at Tier 2+ ($600-$3K/mo). Architecture decisions, not cost decisions.*